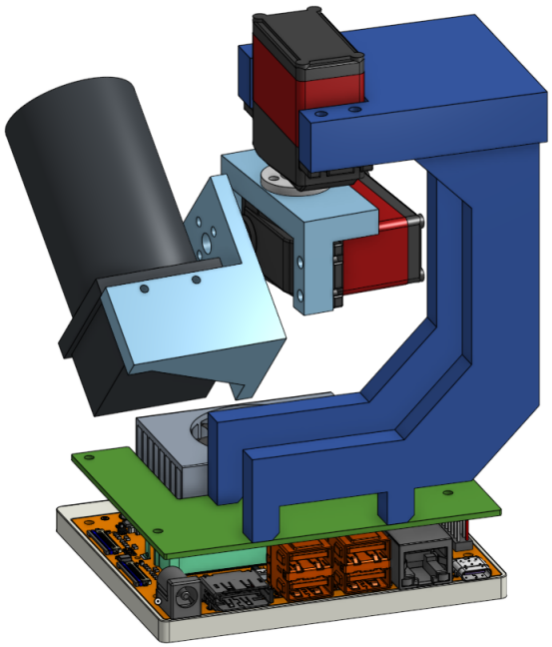
Team Contributions: Rev 0
RoCam

Team #3, SpaceY
Zifan Si
Jianqing Liu
Mike Chen
Xiaotian Lou

January 28, 2026

This document summarizes the contributions of each team member for the Rev 0 Demo. The time period of interest is the time between the PoC demo and the Rev 0 demo; the contributions prior to the PoC are NOT included.

1 Demo Plans



We will present the Rev 0 demonstration according to the plan in the Development Plan document. One of the team members will be in charge of the demonstration.

2 Team Meeting Attendance

Student	Meetings
Total	14
Xiaotian Lou	14
Zifan Si	14
Jianqing Liu	14
Shike Chen	14

3 Supervisor/Stakeholder Meeting Attendance

Supervisor's Name: Sirouspour, Shahin

Student	Meetings
Total	1
Xiaotian Lou	1
Zifan Si	1
Jianqing Liu	1
Shike Chen	1

4 Lecture Attendance

Student	Lectures
Total	1
Xiaotian Lou	1
Zifan Si	0
Jianqing Liu	0
Shike Chen	0

5 TA Document Discussion Attendance

TA's Name: Tanya Djavaherpour

Student	Lectures
Total	1
Xiaotian Lou	1
Zifan Si	1
Jianqing Liu	1
Shike Chen	1

6 Commits

Student	Commits	Percent
Total	198	100%
Xiaotian Lou	22	11.1%
Zifan Si	65	32.8%
Jianqing Liu	85	42.9%
Shike Chen	26	13.13%

Notes. Commit counts reflect merges to the `main` branch only. During the Rev 0 window, Xiaotian Lou focused primarily on model training and on developing a Jetson-based TensorRT benchmarking pipeline, and Shike Chen focused primarily on the labeling application. Much of this work was device-centric (on-target iterations, data/tooling validation, and benchmark runs) and lived in experimental branches, local device workspaces, or separate work-in-progress changes that were not yet ready to merge into `main`. As a result, their `main`-branch commit totals under-represent the effort invested. Integration-ready components and summaries will be upstreamed through PRs as the implementations stabilize.

7 Issue Tracker

Student	Authored (O+C)	Assigned (C only)
Xiaotian Lou	2	5
Zifan Si	2	7
Jianqing Liu	29	22
Shike Chen	11	10

Notes.

- **Counting rule:** "Authored (O+C)" counts issues opened by a member (open or closed). "Assigned (C only)" counts issues where the member was the assignee at closure.
- **Role-based workflow:** During the Rev 0 period, Jianqing Liu (project manager, "Pega") centrally opened, assigned, and closed most issues based on team decisions captured in meeting notes; hence his "Authored" count is higher.
- **Credit vs. logistics:** Centralized opening/closing is administrative. Actual implementation credit is reflected in commits, PRs, and code reviews; an issue can have multiple contributors beyond the final assignee.
- **Assignment semantics:** Ownership may change during an issue's lifetime; we attribute "Assigned (C only)" to the final assignee at closure to represent delivery responsibility.

8 CICD

We use GitHub Actions with five workflows that cover documentation, firmware, backend, and the operator UI.

build-tex (push to `main`, `docs/**`, or manual) Compiles LaTeX with TeX Live 2025 on Ubuntu and commits the built PDFs back to the repository. The job runs `make -B` under `docs/` and then auto-commits any updated `.pdf` files (`contents: write`).

check-tex (pull requests touching `docs/**`) Validates that the LaTeX compiles on PRs, uploads any changed PDFs as an artifact, then formats all `.tex` sources with `latexindent` (80-column wrapping) and commits the formatting changes. This keeps the documentation reproducible and consistently formatted.

firmware-ci (push/PR under `src/pcb-firmware/**`) Builds the embedded firmware with Cargo on Ubuntu and uploads the resulting binary as an artifact. The job also enforces Rust source formatting via `cargo fmt --check`.

backend-ci (pull requests touching `src/backend/**`) Runs backend lint checks on Ubuntu using Python 3.10 with pip dependency caching. The workflow installs dependencies from `src/backend/requirement.txt` (excluding any pinned `pip` line) and performs a basic syntax check via `python -m compileall -q .` in `src/backend`. This helps catch Python syntax errors early before merging.

react-ci (push/PR for the React app) Builds and smoke-tests the operator UI on Node 20. Dependencies are installed with `npm ci`; a lightweight `ci:smoke` runs on every change. Unit tests and production build currently run in *best-effort* mode (non-blocking), and any build artifacts are uploaded when present.

By Rev 0 close, we will deliver substantially deeper, React-focused UI tests (components, hooks, interactions) if we finish full stack intergration.

- **Component tests** with React Testing Library + Jest/Vitest: role-based queries (no test-ids), user interactions via `@testing-library/user-event`, keyboard/focus flows, and accessibility checks with `jest-axe`.
- **State & hooks** coverage: test reducers, Context providers, and custom hooks (e.g., `useGimbal`, `useVideo`) using `@testing-library/react` and `@testing-library/react-hooks`; assert stable re-renders and memoization (`React.memo`, `useCallback`, `useMemo`).
- **Coverage gates**: enforce $\geq$80% lines/branches on UI packages; coverage upload and `--coverageReporters` in CI; mark unit+integration jobs as *required*.
- **Linting/consistency**: `eslint` with `eslint-plugin-react` and `eslint-plugin-jsx-a11y`; `prettier --check` as a required gate.

Near-term roadmap (to be enforced via branch protection)

- **Formatting gates:** add `black --check` for Python sources and `prettier --check` for the React codebase. If formatting is not clean, the PR will fail and be blocked from merging.
- **Cross-platform test matrix:** add a Python test job that runs the full unit test suite (15 tests) on {`ubuntu-latest`, `macos-latest`, `windows-latest`} $\times$ Python {3.9, 3.10, 3.11, 3.12, 3.13}. The job will install dependencies and execute the tests; failures will block merges.

9 Team Charter Trigger Items

Quantified Triggers (per Team Charter)

- **Meeting cadence & attendance:** weekly team meetings; prior notice required for any absence; target attendance $\geq$85%.
- **Preparation & quality:** come to meetings with concise updates, blockers, and options; maintain decision logs and lightweight RACI notes.
- **PR/issue hygiene:** every task has a GitHub issue with an owner and estimate; all code changes go through PRs that reference an issue; no direct pushes to `main`.
- **Review SLA & CI gates:** at least one reviewer approval; address review comments within 7 days (unless a different SLA is agreed); CI must be green (docs build, firmware build, UI smoke).

- **Documentation standards:** LaTeX compiles reproducibly on CI; PDFs are published on `main`; `latexindent` enforces consistent formatting on PRs.
- **Runtime KPIs (demo readiness):** track end-to-end latency (p50/p95), FPS, target re-acquisition time, crash-free session rate, and UI task success; for the PoC demo we target ∼100–500 ms latency, ≥15 FPS, and ≤2 s re-acquisition.
- **Underperformance trigger:** if a member underdelivers for >3 weeks, initiate pairing/guardrails and, if needed, escalate to TA/instructor.

Violations Observed During the Rev 0 Period

None observed. All members met the charter triggers: weekly meetings were held with advance notice for any conflicts; PRs referenced issues and passed CI; review comments were addressed within the 7-day SLA; documentation built reproducibly; runtime KPIs were recorded for the demo.

Plan to Sustain Compliance

- Keep branch protection (no direct pushes to `main`; ≥1 review; CI required).
- Promote formatting to required checks: `black --check` (Python) and `prettier --check` (React).
- Add a cross-platform Python test matrix (Ubuntu/macOS/Windows; Python 3.9–3.13) as a required status check.
- Use a meeting-issue template (owner, ETA, blockers) and maintain the decision log to preserve cadence and traceability.
- Clarify KPI thresholds for upcoming milestones and monitor p50/p95 latency, FPS, and re-acquisition time on each demo run.

10 Additional Productivity Metrics

At this time, we have no additional productivity metrics to report.